ASSIGMENT-17.1

NAME: G. LIKHITHA RAO

BATCH: 35

HALL TICKET NUMBER: 2303A52487

Task 1 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

• Handle missing values in columns (age, income, city).

• Remove duplicate records.

• Standardize text values (e.g., city names in lowercase).

• Encode categorical variables (gender, city).

Sample Code:

```
import pandas as pd
data = pd.read_csv("shopping_trends.csv")
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset?select=shopping_trends.csv

Expected Output:

• A cleaned Pandas data frame ready for analysis.

CODE:

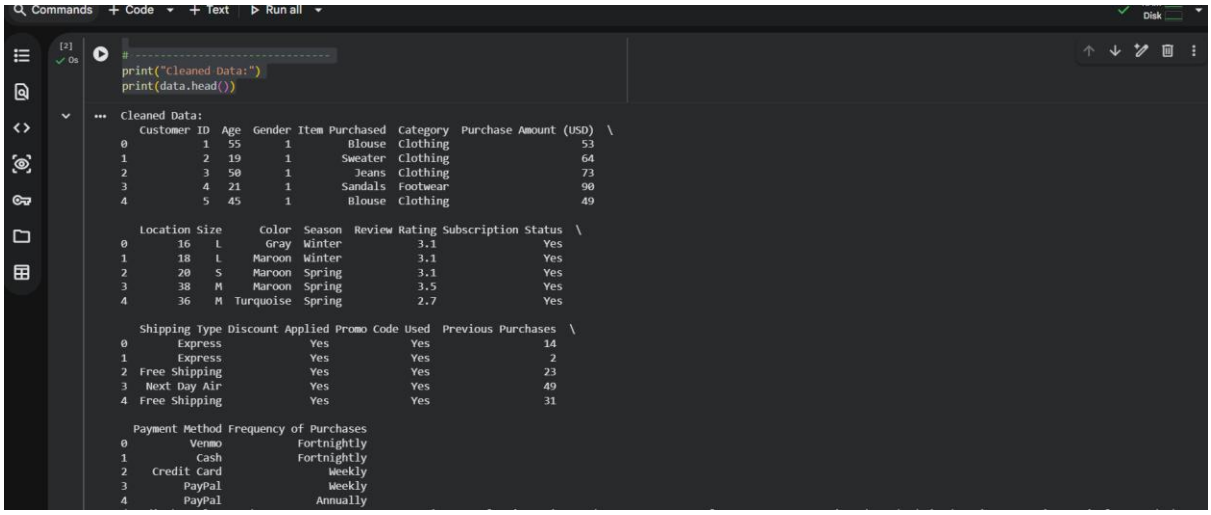
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
data = pd.read_csv("/content/shopping_trends_updated.csv")
```

```
# --------------------------------
# 1. Handle Missing Values
# --------------------------------
# Age & Income → fill with median
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Purchase Amount (USD)'].fillna(data['Purchase Amount (USD)'].median(),
inplace=True)

# City → fill with mode
data['Location'].fillna(data['Location'].mode()[0], inplace=True)

# --------------------------------
# 2. Remove Duplicates
# --------------------------------
data.drop_duplicates(inplace=True)

# --------------------------------
# 3. Standardize Text
# --------------------------------
data['Location'] = data['Location'].str.lower()

# --------------------------------
# 4. Encode Categorical Variables
# --------------------------------
le = LabelEncoder()
data['Gender'] = le.fit_transform(data['Gender'])
data['Location'] = le.fit_transform(data['Location'])

# --------------------------------
```

print("Cleaned Data:")

print(data.head())

OUTPUT:



```
Cleaned Data:
   Customer ID  Age  Gender Item Purchased  Category  Purchase Amount (USD)  \
0            1   55       1         Blouse  Clothing                     53
1            2   19       1        Sweater  Clothing                     64
2            3   50       1          Jeans  Clothing                     73
3            4   21       1        Sandals  Footwear                     90
4            5   45       1         Blouse  Clothing                     49

   Location Size      Color  Season  Review Rating Subscription Status  \
0       16    L       Gray  Winter           3.1                  Yes
1       18    L     Maroon  Winter           3.1                  Yes
2       20    S     Maroon  Spring           3.1                  Yes
3       38    M     Maroon  Spring           3.5                  Yes
4       36    M  Turquoise  Spring           2.7                  Yes

   Shipping Type Discount Applied Promo Code Used  Previous Purchases  \
0       Express              Yes             Yes                  14
1       Express              Yes             Yes                   2
2  Free Shipping             Yes             Yes                  23
3   Next Day Air             Yes             Yes                  49
4  Free Shipping             Yes             Yes                  31

  Payment Method Frequency of Purchases
0          Venmo            Fortnightly
1           Cash            Fortnightly
2    Credit Card                 Weekly
3         PayPal                 Weekly
4         PayPal               Annually
```

Task 2 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Instructions:

• Convert date columns into datetime format.

• Create new features (month, year, day of week).

• Normalize sales amount column.

• Detect and handle outliers in dataset.

Sample Code:

import pandas as pd

data = pd.read_csv("sales_data.csv")

print(data.head())

Dataset link:

https://www.kaggle.com/datasets/vinothkannaece/sales-

dataset?select=sales_data.csv

Expected Output:

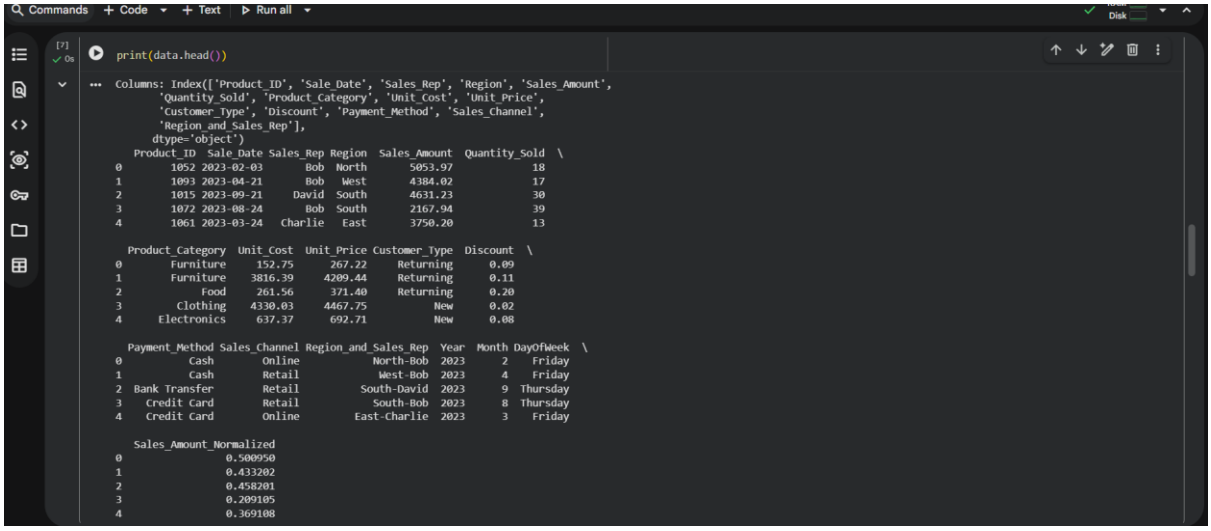
• A transformed dataset with time-based features and

normalized values

CODE:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

data = pd.read_csv("/content/sales_data.csv")

# Clean column names
data.columns = data.columns.str.strip()

print("Columns:", data.columns)

# Replace 'Date' with your actual column name
date_col = 'Sale_Date'   # <-- change if needed

# Convert to datetime
data[date_col] = pd.to_datetime(data[date_col])

# Feature extraction
data['Year'] = data[date_col].dt.year
data['Month'] = data[date_col].dt.month
data['DayOfWeek'] = data[date_col].dt.day_name()

# Normalize (change column if needed)
scaler = MinMaxScaler()
data['Sales_Amount_Normalized'] = scaler.fit_transform(data[['Sales_Amount']])

print(data.head())
```

OUTPUT:



```
print(data.head())

Columns: Index(['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount',
       'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price',
       'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel',
       'Region_and_Sales_Rep'],
      dtype='object')
   Product_ID  Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
0        1052 2023-02-03       Bob  North       5053.97             18
1        1093 2023-04-21       Bob   West       4384.02             17
2        1015 2023-09-21     David  South       4631.23             30
3        1072 2023-08-24       Bob  South       2167.94             39
4        1061 2023-03-24   Charlie   East       3750.20             13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0        Furniture     152.75      267.22     Returning      0.09
1        Furniture    3816.39     4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
3         Clothing    4330.03     4467.75           New      0.02
4      Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep  Year  Month DayOfWeek  \
0           Cash        Online           North-Bob  2023      2    Friday
1           Cash        Retail            West-Bob  2023      4    Friday
2  Bank Transfer        Retail          South-David  2023      9  Thursday
3    Credit Card        Retail            South-Bob  2023      8  Thursday
4    Credit Card        Online         East-Charlie  2023      3    Friday

   Sales_Amount_Normalized
0                 0.500950
1                 0.433202
2                 0.458201
3                 0.209105
4                 0.369108
```

Task 3 – Healthcare Data Preparation

Task:

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

• Handle missing values in blood_pressure, cholesterol.

• Encode categorical features

• Scale numerical features (min-max or standard scaling).

• Split dataset into training and testing sets.

Dataset link :

https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset

Expected Output:

• A preprocessed dataset ready for machine learning models.

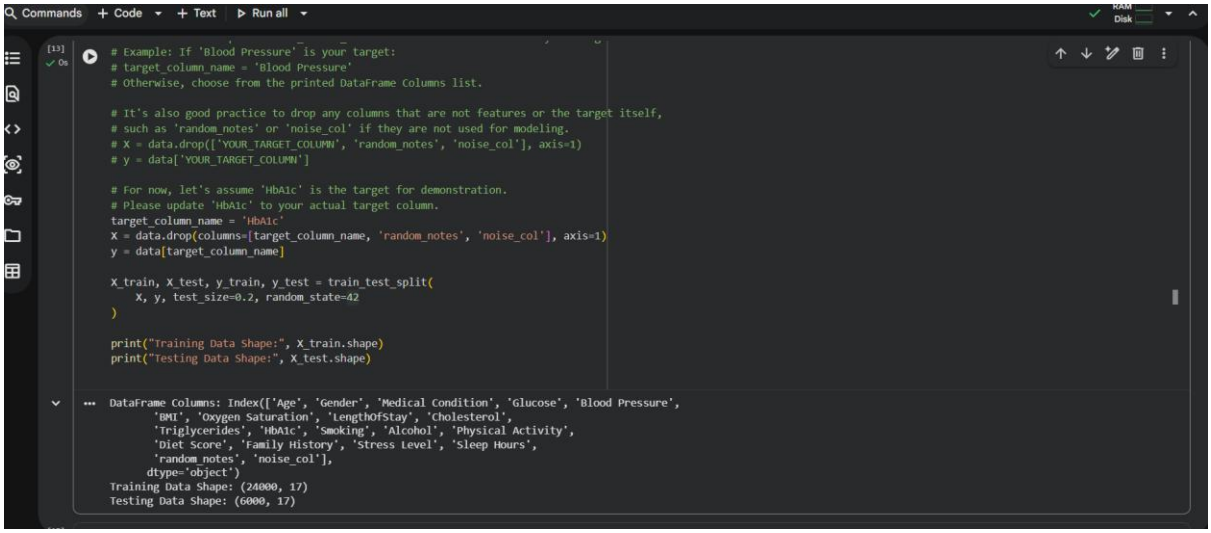
CODE:

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.model_selection import train_test_split
```

```
# Load dataset
data = pd.read_csv("/content/dirty_v3_path.csv")

# --------------------------------
# 1. Handle Missing Values
# --------------------------------
data['Blood Pressure'] = data['Blood Pressure'].fillna(data['Blood Pressure'].mean())
data['Cholesterol'] = data['Cholesterol'].fillna(data['Cholesterol'].mean())

# --------------------------------
# 2. Encode Categorical Columns
# --------------------------------
le = LabelEncoder()
for col in data.select_dtypes(include='object').columns:
    data[col] = le.fit_transform(data[col])

# --------------------------------
# 3. Scale Numerical Features
# --------------------------------
scaler = StandardScaler()
num_cols = data.select_dtypes(include=['int64', 'float64']).columns
data[num_cols] = scaler.fit_transform(data[num_cols])

# --------------------------------
# 4. Train-Test Split
# --------------------------------

# Print columns to help identify the target variable
print("DataFrame Columns:", data.columns)
```

```
# !!! IMPORTANT: Replace 'YOUR_TARGET_COLUMN' with the actual name of your target column !!!
# Example: If 'Blood Pressure' is your target:
# target_column_name = 'Blood Pressure'
# Otherwise, choose from the printed DataFrame Columns list.

# It's also good practice to drop any columns that are not features or the target itself,
# such as 'random_notes' or 'noise_col' if they are not used for modeling.
# X = data.drop(['YOUR_TARGET_COLUMN', 'random_notes', 'noise_col'], axis=1)
# y = data['YOUR_TARGET_COLUMN']

# For now, let's assume 'HbA1c' is the target for demonstration.
# Please update 'HbA1c' to your actual target column.
target_column_name = 'HbA1c'
X = data.drop(columns=[target_column_name, 'random_notes', 'noise_col'], axis=1)
y = data[target_column_name]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

OUTPUT:



Task 4 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

• Handle missing values

• Detect and remove salary outliers.

• Standardize department names (lowercase).

• Apply label encoding to job location, department.

Dataset link:

https://www.kaggle.com/code/vishantmathur/employee-salary

Expected Output:

• A structured dataset ready for HR analytics

CODE:

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder


# Load dataset

data = pd.read_csv("/content/Employee_Salary_Dataset.csv")
```

```
# Print columns to verify
print("DataFrame Columns:", data.columns)

# --------------------------------
# 1. Handle Missing Values
# --------------------------------
data.fillna(method='ffill', inplace=True)

# --------------------------------
# 2. Remove Salary Outliers
# --------------------------------
Q1 = data['Salary'].quantile(0.25)
Q3 = data['Salary'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

data = data[(data['Salary'] >= lower) & (data['Salary'] <= upper)]

# --------------------------------
# 3. Standardize Text
# --------------------------------
# Removed operations for 'Department' as the column does not exist.

# --------------------------------
# 4. Encode Categorical Variables
# --------------------------------
```

```
le = LabelEncoder()
# Removed operations for 'Department' and 'Job_Location' as the columns do not exist.
# If there are other categorical columns, add them here. For example:
# data['Gender'] = le.fit_transform(data['Gender'])


# ------------------------------
print("Processed Employee Data:")
print(data.head())
```

OUTPUT: